Analisi e confronto degli algoritmi di Eulero e Eulero-Cromer con applicazione al pendolo semplice

Autore:

Alberto Demarco

Sotto la guida di

Prof. Giuseppe Magnifico

Dipartimento Interuniversitario di Fisica "Michelangelo Merlin"

Università degli Studi di Bari "Aldo Moro"

A.A. 2025/2026

Indice

1 Introduzione **1**

2 Grafici e Risultati **2**

3 Discussione dei Risultati **4**

Abstract

L'obiettivo dell'esperienza è lo studio, tramite l'implementazione in codice Python, degli algoritmi di Eulero ed Eulero-Cromer, con applicazione al caso del pendolo semplice.

1 Introduzione

Il pendolo semplice è costituito da una massa puntiforme m sospesa tramite un filo inestensibile di lunghezza L. Applicando la seconda legge di Newton alla componente tangenziale del moto, si ottiene l'equazione del moto:

$$\frac{d^2\theta}{dt^2} = -\frac{g}{L}\sin(\theta) \tag{1}$$

dove θ è l'angolo rispetto alla verticale e g è l'accelerazione di gravità.
Per trasformare questa equazione differenziale ordinaria (ODE) del secondo ordine in un sistema di due ODE del primo ordine, introduciamo la velocità angolare $\omega = \frac{d\theta}{dt}$:

$$\frac{d\theta}{dt} = \omega \tag{2}$$

$$\frac{d\omega}{dt} = -\frac{g}{L}\sin(\theta) \tag{3}$$

L'energia meccanica totale del sistema è calcolabile tramite l'equazione $E = \frac{1}{2}mL^2\omega^2 + mgL(1-\cos\theta)$. In assenza di attrito, l'energia meccanica deve essere conservata, divenendo così un importante criterio per valutare la qualità dei metodi numerici applicati.

Soluzione analitica per piccole oscillazioni

Nel limite di piccole oscillazioni ($\sin(\theta) \approx \theta$), l'equazione diventa quella di un oscillatore armonico avente frequenza angolare naturale $\omega_0 = \sqrt{\frac{g}{L}}$. La soluzione analitica in questo specifico caso è nota:

$$\theta(t) = \theta_0\cos(\omega_0 t) + \frac{\omega_{init}}{\omega_0}\sin(\omega_0 t) \tag{4}$$

dove θ_0 è l'angolo iniziale e ω_{init} è la velocità angolare iniziale.

Metodi di Integrazione Numerica

Il metodo di Eulero è il più semplice metodo numerico per la risoluzione di equazioni differenziali ordinarie. Data un'equazione della forma $\frac{dy}{dt} = f(t, y)$, il metodo approssima la derivata con un rapporto incrementale in avanti: $y_{n+1} = y_n + h \cdot f(t_n, y_n)$, dove h indica il passo temporale. Per il sistema del pendolo semplice, le equazioni di aggiornamento calcolate all'istante t_n risultano essere:

$$\theta_{n+1} = \theta_n + h \cdot \omega_n \tag{5}$$

$$\omega_{n+1} = \omega_n - h \cdot \frac{g}{L}\sin(\theta_n) \tag{6}$$

A fini comparativi viene impiegato anche il metodo di Eulero-Cromer, noto per essere una variante simplettica del metodo di Eulero. La differenza fondamentale risiede nell'aggiornare la velocità in primis, utilizzandone poi il nuovo valore per aggiornare la posizione spaziale:

$$\omega_{n+1} = \omega_n - h \cdot \frac{g}{L} \sin(\theta_n) \tag{7}$$

$$\theta_{n+1} = \theta_n + h \cdot \omega_{n+1} \tag{8}$$

Nonostante questa sia un'apparentemente piccola modifica, essa ha conseguenze profonde: questo metodo è a tutti gli effetti un integratore simplettico che riesce a conservare in modo approssimato l'energia del sistema su tempi lunghi, compensando una minor accuratezza analitica rispetto a metodi numerici di ordine superiore.

2 Grafici e Risultati

In questa sezione si riportano i grafici ottenuti dall'implementazione in Python degli algoritmi, accompagnati dai dati numerici estratti per la validazione quantitativa dei modelli.
I valori registrati per l'errore globale (confronto con la soluzione analitica nel regime di piccole oscillazioni) e per la variazione percentuale dell'energia meccanica (su una simulazione di 100 periodi) sono i seguenti:

- **Errore globale medio (Eulero):** 0.0443
- **Errore globale massimo (Eulero):** 0.1563
- **Errore globale medio (Eulero-Cromer):** 0.0006
- **Errore globale massimo (Eulero-Cromer):** 0.0020
- **Variazione percentuale dell'energia (Eulero):** 7786.70%
- **Variazione percentuale dell'energia (Eulero-Cromer):** 1.04%

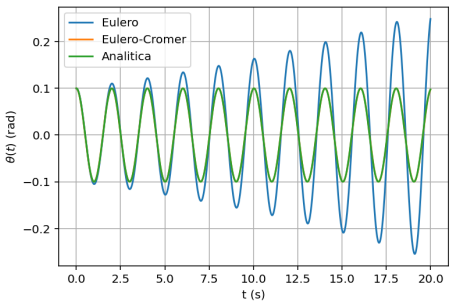
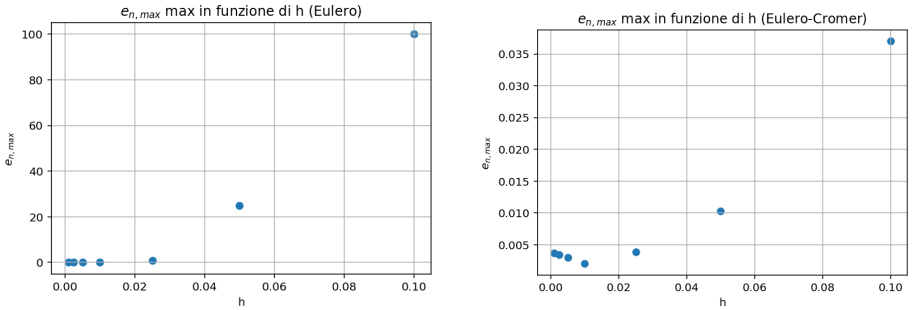


Figura 1: Esercizio 1: Confronto tra le soluzioni numeriche (Eulero ed Eulero-Cromer) e la soluzione analitica per piccole oscillazioni ($\theta_0 = 0.1$ rad). Si nota come la curva di Eulero-Cromer sia pressoché sovrapponibile a quella analitica.



(a) Metodo di Eulero

(b) Metodo di Eulero-Cromer

Figura 2: Esercizio 1: Andamento dell'errore globale massimo al variare del passo temporale h.

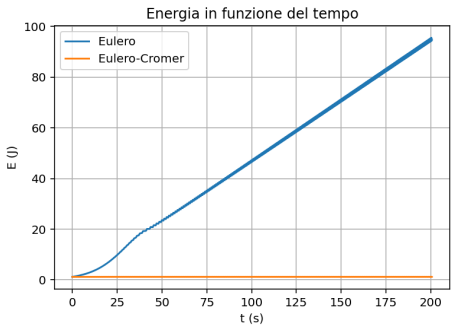
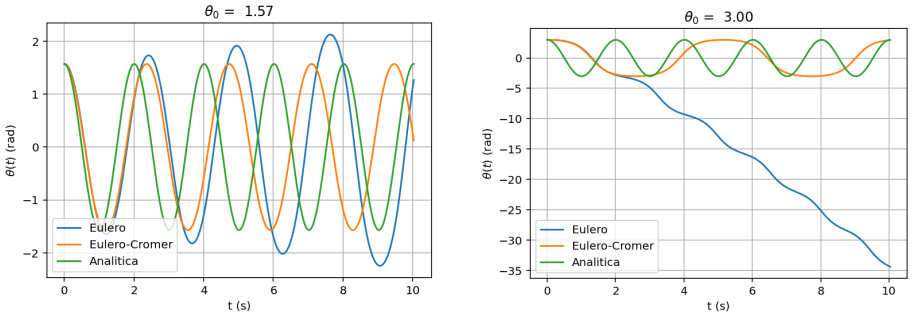


Figura 3: Esercizio 2: Conservazione dell'energia meccanica valutata su 100 periodi ($\theta_0 = 0.5$ rad). Eulero diverge vistosamente, mentre Eulero-Cromer mantiene l'energia limitata attorno al valore iniziale.



(a) $\theta_0 = \pi/2$ rad (90°)

(b) $\theta_0 = 3$ rad ($\approx 172°$)

Figura 4: Esercizio 3: Grandi oscillazioni. Discrepanza tra l'integrazione numerica esatta (Eulero-Cromer) e il modello armonico analitico all'aumentare dell'angolo iniziale.

3 Discussione dei Risultati

Validazione con la soluzione analitica ed errore

Dall'analisi della simulazione per piccole oscillazioni (Figura 1), emerge la netta superiorità del metodo di Eulero-Cromer. Come riportato nei dati iniziali, l'algoritmo di Eulero esplicito accumula un errore globale massimo di 0.1563, mentre l'errore di Eulero-Cromer risulta inferiore di quasi due ordini di grandezza (0.0020). Inoltre, i grafici in Figura 2 confermano la dipendenza dell'errore globale dal passo temporale h: rimpicciolendo il passo di integrazione, la precisione di entrambi i metodi migliora progressivamente, riducendo l'errore commesso.

Conservazione dell'energia meccanica

Il limite principale del metodo di Eulero classico emerge drammaticamente valutando l'evoluzione del sistema su 100 periodi (Figura 3). Poiché Eulero non è un integratore simplettico, introduce un'energia artificiale nel sistema: dai calcoli si registra infatti un aumento spropositato del 7786.70%, evidenziando un comportamento totalmente non-fisico. Al contrario, il metodo di Eulero-Cromer oscilla mantenendosi fedele al valore dell'energia iniziale, accumulando dopo lo stesso numero di periodi una variazione percentuale quasi trascurabile (appena il 1.04%). Questo certifica l'efficacia di Eulero-Cromer per lo studio di dinamiche conservative su tempi lunghi.

Il limite del caso armonico (Grandi oscillazioni)

I risultati riportati in Figura 4 rispondono al quesito riguardante la validità della soluzione analitica per ampiezze maggiori. Per $\theta_0 = \pi/2$ rad e, in maniera ancora più marcata, per $\theta_0 = 3$ rad, la soluzione analitica perde progressivamente di validità. Ciò accade perché l'approssimazione $\sin(\theta) \approx \theta$ non è più applicabile. A grandi angoli, il valore esatto di $\sin(\theta)$ è minore del valore in radianti di θ; di conseguenza, la componente reale della forza di gravità tangenziale che richiama il pendolo è più debole rispetto a quella ipotizzata dal modello armonico. Il pendolo "reale" (simulato numericamente) subisce quindi un'accelerazione minore e impiega più tempo a completare un'oscillazione. Rispetto al caso armonico, il periodo aumenta visibilmente, causando il netto sfasamento temporale osservabile nei grafici.

Appendice: Codice Python

Di seguito si riporta il codice completo utilizzato per le simulazioni dell'esercitazione.

```python

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Mon Mar 16 09:57:22 2026

@author: alberto
"""

import numpy as np
from matplotlib import pyplot as plt

#Esercizio 1

def eulero(t, h, theta_0, omega_in, L, g):

    theta = np.zeros(len(t))  #def gli array pieni di zeri
    omega = np.zeros(len(t))

    theta[0] = theta_0   # def i valori iniziali per theta e omega
    omega[0] = omega_in

    for i in range(len(t) - 1):

        theta[i+1] = theta[i] + h*omega[i]
        omega[i+1] = omega[i] - h*(g/L)*np.sin(theta[i])


    return theta, omega #restituisce una tupla


def eulero_cromer(t, h, theta_0, omega_in, L, g):

    theta = np.zeros(len(t))  #def gli array pieni di zeri
    omega = np.zeros(len(t))

    theta[0] = theta_0   # def i valori iniziali per theta e omega
    omega[0] = omega_in

    for i in range(len(t) - 1):

        omega[i+1] = omega[i] - h*(g/L)*np.sin(theta[i])
        theta[i+1] = theta[i] + h*omega[i+1]



    return theta, omega

#def le costanti da usare

h = 0.01 #s
t_max = 20.0 #s
L = 1 #m
theta_0 = 0.1 #rad
omega_in = 0.0 #rad/s
g = 9.81 #m/s^2
t_0 = 0.0 #s
```

```python
w_0 = np.sqrt(g/L) #s^-1


t = np.arange(t_0, t_max, h)


#def array per i due metodi + confronto sol analitica

theta_eulero, omega_eulero = eulero(t, h, theta_0, omega_in, L, g)

theta_eulero_cromer, omega_eulero_cromer = eulero_cromer(t, h, theta_0, omega_in, L, g)

theta_analitica = theta_0*np.cos(w_0*t) + (omega_in/w_0)*np.sin(w_0*t)

#blocco di plot

plt.plot(t, theta_eulero, label="Eulero")
plt.plot(t, theta_eulero_cromer, label="Eulero-Cromer")
plt.plot(t, theta_analitica, label="Analitica")

plt.xlabel("t (s)")
plt.ylabel(r"$\theta (t)$ (rad)")
plt.legend()
plt.grid(True)

plt.show()


#errori


e_glob_theta_eulero = np.abs(theta_analitica - theta_eulero) # vettore degli errori di discretizzazione globale al passo i-esimo
e_medio_theta_eulero = np.mean(e_glob_theta_eulero) #errore medio
e_max_theta_eulero = np.max(e_glob_theta_eulero)


print("Errore globale medio (Eulero): %.4f" % e_medio_theta_eulero)
print("Errore globale massimo (Eulero) : %.4f" % e_max_theta_eulero)

e_glob_theta_eulero_cromer = np.abs(theta_analitica - theta_eulero_cromer) #vettore degli errori di discretizzazione globale al passo i-esimo
e_medio_theta_eulero_cromer = np.mean(e_glob_theta_eulero_cromer) # errore medio
e_max_theta_eulero_cromer = np.max(e_glob_theta_eulero_cromer)

print("Errore globale medio (Eulero-Cromer): %.4f" % e_medio_theta_eulero_cromer)
print("Errore globale massimo (Eulero-Cromer) : %.4f" % e_max_theta_eulero_cromer)


#variazione di h: plot di e(h)


h_values = np.array([0.1, 0.05, 0.025, 0.01, 0.005,0.0025, 0.001])

```

```python
errori_max_values_eulero = []
errori_max_values_eulero_cromer = []

for h in h_values:   #ciclo sui valori di h

    t = np.arange(t_0, t_max, h) #ricalcolo il vettore dei tempi

    #ricalcolo le varie theta

    theta_eulero = eulero(t, h, theta_0, omega_in, L, g)[0]
    theta_eulero_cromer = eulero_cromer(t, h, theta_0, omega_in, L, g)[0]

    theta_analitica = theta_0*np.cos(w_0*t) + (omega_in/w_0)*np.sin(w_0*t)

    #calcolo vettore dell'errore globale al passo n

    errore_eulero = np.abs(theta_analitica - theta_eulero)
    errore_eulero_cromer = np.abs(theta_analitica - theta_eulero_cromer)

    e_max_eulero = np.max(errore_eulero)
    e_max_eulero_cromer = np.max(errore_eulero_cromer)

    #append del valore massimo dell'errore all'array degli errori

    errori_max_values_eulero.append(e_max_eulero)
    errori_max_values_eulero_cromer.append(e_max_eulero_cromer)

#plot

plt.title("$e_{n, max}$ max in funzione di h (Eulero)")
plt.scatter(h_values, errori_max_values_eulero, marker="o")
plt.xlabel("h")
plt.ylabel(r"$e_{n, max}$")
plt.grid(True)
plt.show()


plt.title("$e_{n, max}$ max in funzione di h (Eulero-Cromer)")
plt.scatter(h_values, errori_max_values_eulero_cromer, marker="o")
plt.xlabel("h")
plt.ylabel(r"$e_{n, max}$")
plt.grid(True)

plt.show()


# ##################################################################


#Esercizio 2

#def le costanti da usare

theta_0 = 0.5

h = 0.01 #s
L = 1 #m
```

```python
omega_in = 0.0 #rad/s
g = 9.81 #m/s^2
t_0 = 0.0 #s
w_0 = np.sqrt(g/L) #s^-1
T = 2*np.pi/w_0

t_max = 100*T

t = np.arange(t_0, t_max, h)

m = 1 #suppongo la massa uguale a 1kg

#energia meccanica

def energia(theta, omega, L, m, g):

    return 0.5*m*(L*omega)**2 + m*L*g*(1 - np.cos(theta))

#def gli array di theta e omega


theta_eulero, omega_eulero = eulero(t, h, theta_0, omega_in, L, g)
theta_eulero_cromer, omega_eulero_cromer = eulero_cromer(t, h,
    theta_0, omega_in, L, g)


#def gli array dell'energia in entrambi i metodi


energia_eulero = energia(theta_eulero, omega_eulero, L, m, g)
energia_eulero_cromer = energia(theta_eulero_cromer,
    omega_eulero_cromer, L, m, g)


#plot

plt.plot(t, energia_eulero, label="Eulero")
plt.plot(t, energia_eulero_cromer, label="Eulero-Cromer")

plt.xlabel("t (s)")
plt.ylabel("E (J)")
plt.title("Energia in funzione del tempo")

plt.ylabel("E (J)")
plt.grid(True)
plt.legend()

plt.show()

#errore percentuale


err_eulero = ((energia_eulero[-1] - energia_eulero[0])/
    energia_eulero[0])*100
err_eulero_cromer = ((energia_eulero_cromer[-1] -
    energia_eulero_cromer[0])/energia_eulero_cromer[0])*100


print("Errore percentuale Eulero: %.5f" % err_eulero)
print("Errore percentuale Eulero-Cromer: %.5f" % err_eulero_cromer)


```

```python

#    ##################################################################

#Esercizio 3

#def le costanti da usare


theta_0 = [np.pi/2, 3]  # due valori per theta_0. conviene salvarli in una lista

h = 0.01 #s
L = 1 #m
omega_in = 0.0 #rad/s
g = 9.81 #m/s^2
t_0 = 0.0 #s
w_0 = np.sqrt(g/L) #s^-1
T = 2*np.pi/w_0

t_max = 5*T

t = np.arange(t_0, t_max, h)


for theta in theta_0:


    theta_eulero, omega_eulero = eulero(t, h, theta, omega_in, L, g)

    theta_eulero_cromer, omega_eulero_cromer = eulero_cromer(t, h, theta, omega_in, L, g)

    theta_analitica = theta*np.cos(w_0*t) + (omega_in/w_0)*np.sin(w_0*t)


    plt.title(fr"$\theta_0$ = {theta: .2f}")
    plt.plot(t, theta_eulero, label="Eulero")
    plt.plot(t, theta_eulero_cromer, label="Eulero-Cromer")
    plt.plot(t, theta_analitica, label="Analitica")


    plt.xlabel("t (s)")
    plt.ylabel(r"$\theta (t)$ (rad)")
    plt.legend()
    plt.grid(True)

    plt.show()
```